

分支结构

主要内容

- **分支结构**

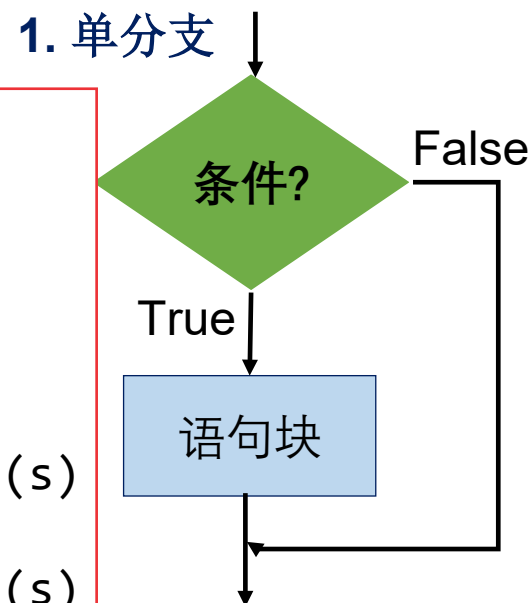
- 单分支/双分支/多分支结构
- 逻辑运算符
- python代码编写规范

条件执行

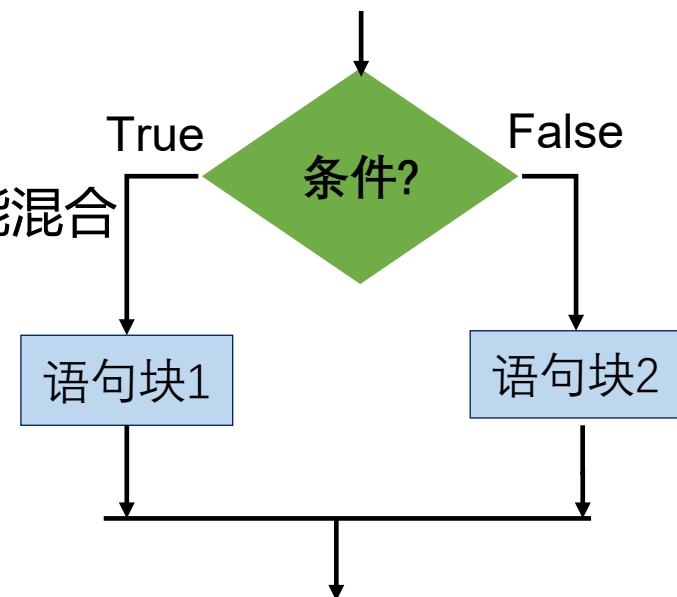
- if 后面为**条件表达式（真值判断）**，**行尾的冒号**表示代码块开始
- 有些语言要求条件表达式用括号包含 (expr)，python并不需要
- (真值判断) 条件满足时，顺序执行代码块中的代码
 - 代码块可与if在同一行（不建议的编程风格）：
 - 一般仅包含一条语句，**拓展：或以分号隔开的多条语句**
 - 这些语句不允许是复合语句
 - 代码块与if不在同一行
 - 代码块**整体缩进相同的位置**
 - 结束缩进**回到if语句开始的位置，表示代码块的结束
 - 建议以**4个空格**为基本缩进单位，不建议采用制表符缩进，更不能混合使用制表符和空格

```
if expr:
    statement
    ...
    statement

if expr:
    statement(s)
else:
    statement(s)
```



2. 双分支



```
gpa = float(input("gpa? "))
if gpa >= 2.0:
    print("Welcome to Mars University!")
else:
    print("Application denied.")
```

```
gpa = float(input("gpa? "))
if gpa >= 2.0:
    print("Application accepted.")
```

```
if gpa >= 2.0: print("...")
if gpa >= 2.0: print("..."); print("...")
```

if/else语句嵌套与多分支结构

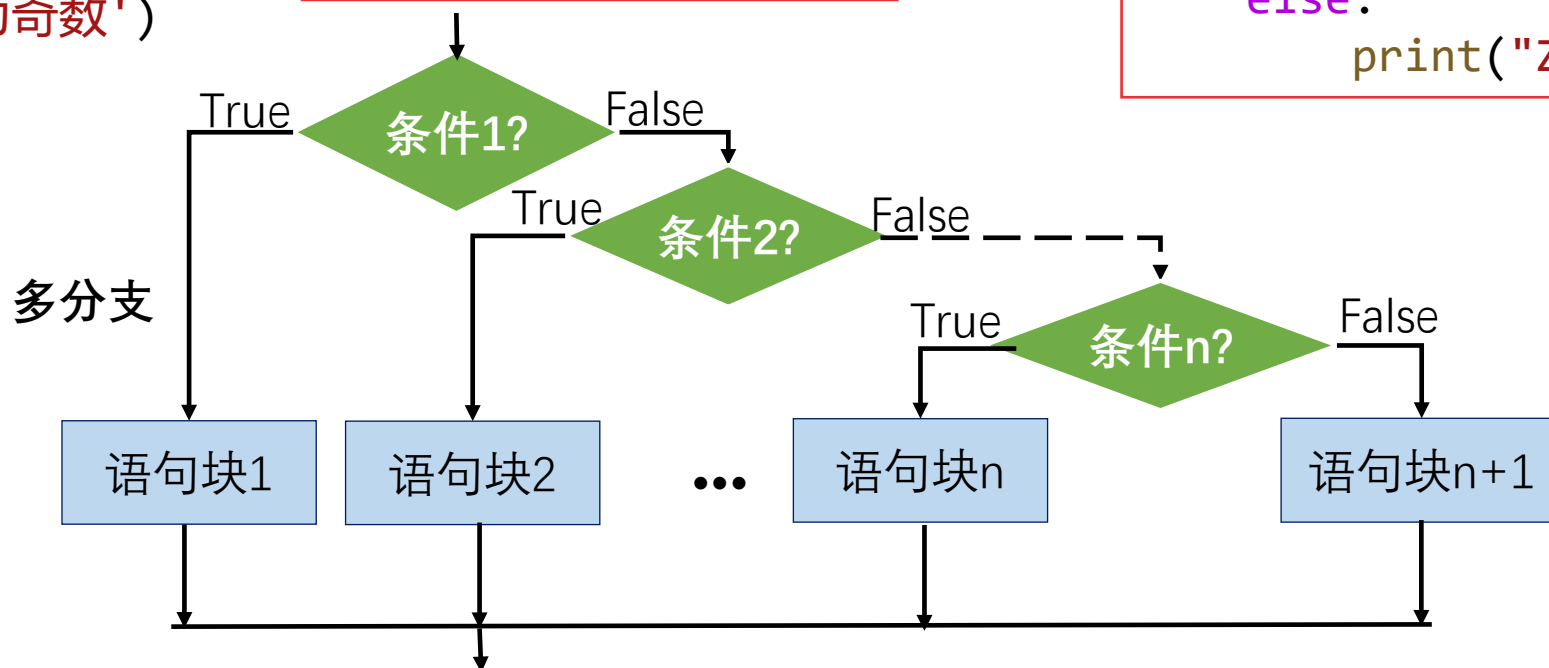
- 如果条件表达式之间是独立的且可以执行语句块的任意组合时，可以采用顺序的多个if语句
- 如果要根据条件表达式决定执行多个语句块中的其中某1个或0个？ 嵌套if语句 ==> 多分支结构

```
a, b, c = 4, 5, 7
if a % 2:
    print(a, '为奇数')
if b % 2:
    print(b, '为奇数')
if c % 2:
    print(c, '为奇数')
```

```
if x > 0:
    print("Positive")
if x < 0:
    print("Negative")
if x == 0:
    print("Zero")
```

```
if x > 0:
    print("Positive")
else:
    if x < 0:
        print("Negative")
    else:
        print("Zero")
```

branch.py



if/else语句嵌套与多分支结构

- 如果条件表达式之间是独立的且可以执行语句块的任意组合时，可以采用顺序的多个if语句
- 如果要根据条件表达式决定执行多个语句块中的其中某1个或0个？ 嵌套if语句 ==> 多分支结构

```
percent = float(input("What percentage did you earn? "))
```

```
if percent >= 90:  
    print("You got an A!")
```

```
if percent >= 80:  
    print("You got a B!")
```

```
if percent >= 70:  
    print("You got a C!")
```

```
if percent >= 60:  
    print("You got a D!")
```

```
if percent < 60:  
    print("You got an F!")
```

上述代码有错误

```
percent = float(input("What percentage did you earn? "))  
if not (0 <= percent <= 100): # percent < 0 or percent > 100  
    raise ValueError('invalid percent')  
if percent >= 90:  
    print("You got an A!")  
else:  
    if percent >= 80:  
        print("You got a B!")  
    else:  
        if percent >= 70:  
            print("You got a C!")  
        else:  
            if percent >= 60:  
                print("You got a D!")  
            else: # 一定小于60  
                print("You got an F!")
```

branch.py

if/else语句嵌套与多分支结构

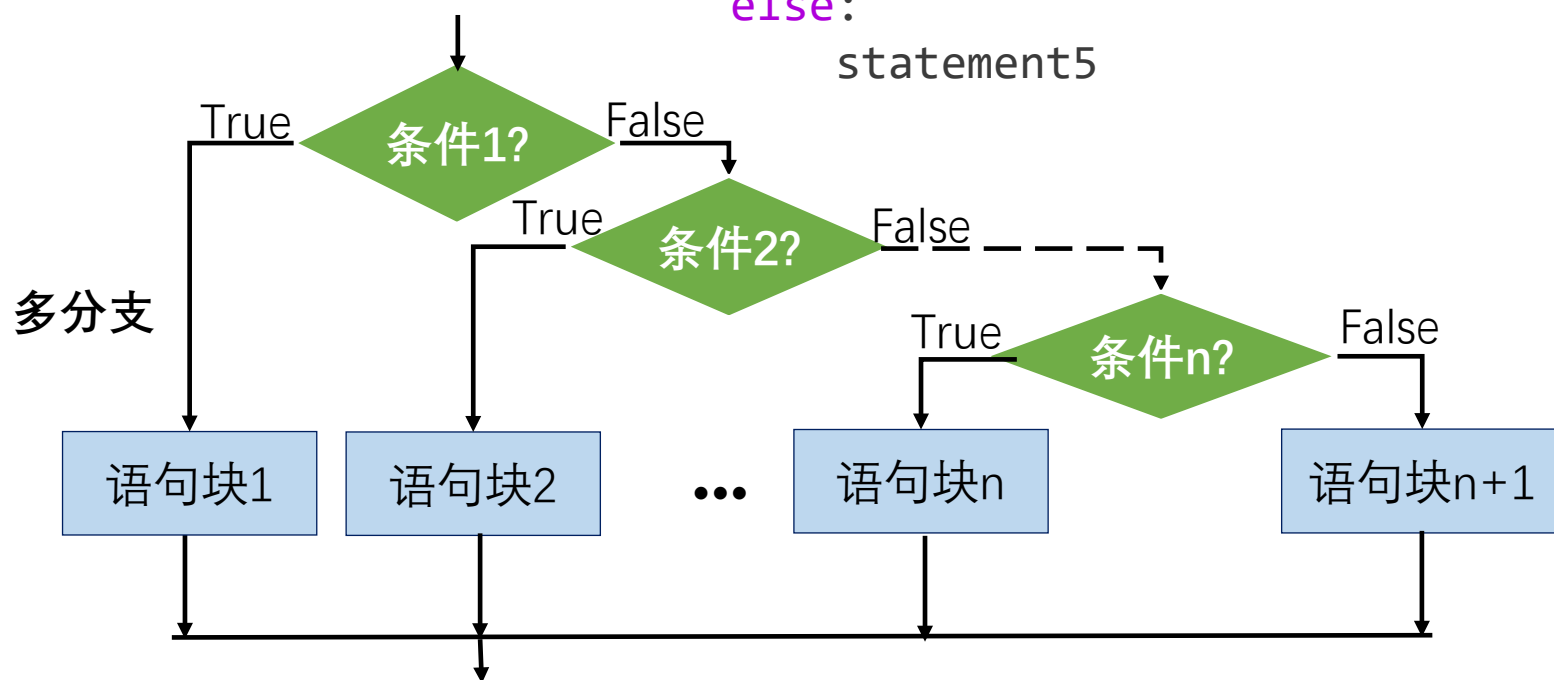
- 多分支结构：根据条件表达式决定执行多个语句块中的其中某1个或0个（不包含else子句）
 - 关键字elif是else if的缩写
 - 条件i 实际上等于 前面的条件都为假 再加上条件i

```
if x > 0:  
    print("Positive")  
else:  
    if x < 0:  
        print("Negative")  
    else:  
        print("Zero")
```

```
if x > 0:  
    print("Positive")  
elif x < 0:  
    print("Negative")  
else:  
    print("Zero")
```

```
if expr1:  
    statement1  
else:  
    if expr2:  
        statement2  
    else:  
        if expr3:  
            statement3  
        else:  
            if expr4:  
                statement4  
            else:  
                statement5
```

```
if expr1:  
    statement1  
elif expr2:  
    statement2  
elif expr3:  
    statement3  
elif expr4:  
    statement4  
else:  
    statement5
```



if/else语句嵌套与多分支结构

- 多分支结构中，条件i实际上等于 前面的条件都为假 再加上条件i
- 注意条件表达式的顺序

```
percent = float(input("What percentage did you earn? "))
if not (0 <= percent <= 100): # percent < 0 or percent > 100
    raise ValueError('invalid percent')
if percent >= 90:
    print("You got an A!")
else:
    if percent >= 80:
        print("You got a B!")
    else:
        if percent >= 70:
            print("You got a C!")
        else:
            if percent >= 60:
                print("You got a D!")
            else: # 一定小于60
                print("You got an F!")
```

```
if percent >= 90:
    print("You got an A!")
elif percent >= 80:
    print("You got a B!")
elif percent >= 70:
    print("You got a C!")
elif percent >= 60:
    print("You got a D!")
else: # percent < 60
    print("You got an F!")
```

下述代码有错，60分以上都是等级D

```
if percent >= 60:
    print("You got a D!")
elif percent >= 80:
    print("You got a B!")
elif percent >= 70:
    print("You got a C!")
elif percent >= 90:
    print("You got an A!")
else: # percent < 60
    print("You got an F!")
```

条件表达式限制

- C语言：下面两种用法都允许，容易混淆

if (a=3) printf("..."), 首先a = 3, 然后if (a) ...

if (a==3) printf("...")

- python不允许在条件表达式中出现赋值运算 =

```
>>> if a=3:pass
SyntaxError: invalid syntax
```

拓展：python3.8引入assignment expression or named expression

name := expression

- := 称为 walrus运算符，该表达式求解后的值就是expression求解后的值，与此同时该值被赋值给name

```
a = 3
if a:
    print(a)

if a:=3:
    print(a)
```

```
>>> a = (b:=4+5) * 7
>>> b
9
>>> a
63
```

```
if text:= input('n = (default 100) '):
    n = int(text)
else:
    n = 100
```

选择结构常见错误：错误缩进

```
import math
radius = -20
if radius >= 0:
    area = radius * radius * math.pi
print("The area is", area)
```



```
import math
radius = -20
if radius >= 0:
    area = radius * radius * math.pi
    print("The area is", area)
```

```
i = 1
j = 2
k = 3
if i > j:
    if i > k:
        print('A')
    else:
        print('B')
```

缩进不一样，
执行的结果也
会不一样!!

左边没有输出
右边打印B

```
i = 1
j = 2
k = 3
if i > j:
    if i > k:
        print('A')
else:
    print('B')
```

if/else结构的代码重构

- 将公共的代码移出if/else结构以删除冗余

branch.py

```
if money < 500:
    print("You have $", money, "left.")
    print("Cash is dangerously low. Bet carefully.")
    bet = int(input("How much do you want to bet?"))
elif money < 1000:
    print("You have $", money, "left.")
    print("Cash is somewhat low. Bet moderately.")
    bet = int(input("How much do you want to bet?"))
else:
    print("You have $", money, "left.")
    print("Cash is in good shape. Bet liberally.")
    bet = int(input("How much do you want to bet?"))
```

```
print("You have $", money, "left.")
if money < 500:
    print("Cash is dangerously low. Bet carefully.")
elif money < 1000:
    print("Cash is somewhat low. Bet moderately.")
else:
    print("Cash is in good shape. Bet liberally.")
bet = int(input("How much do you want to bet?"))
```

条件表达式

出现在if/while中作为决定分支走向或循环结束的条件的条件表达式expr

- 一般包括比较运算符，运算的结果是True/False
- 也可以是任何表达式，会进行真值判断 `bool(expr)`
- 基本规则： 一个对象一般真值判断为真，除非：
 - 如果该对象包含了__bool__方法，由调用该方法的返回值确定
 - 如果该对象包含了__len__方法，调用该方法，如果返回0，则真值判断为假
- **真值判断为False的对象**： False/None/值为0的数值对象/长度为0的空序列对象(比如空字符串)
- 真值判断为True的对象： True/**非0数值对象、非空序列对象**

```
if __name__ == '__main__':  
    main()  
if 2: # 相当于 if bool(2)  
    print('真值判断为真'))
```

```
if i != 0:  
    print(i, '!=0')  
  
if len(s) != 0:  
    print(s)  
  
# pythonic version  
if i:  
    print(i, '!=0')  
  
if s:  
    print(s)
```

真值判断为

假: 0 0.0 0j " [] () set() {} range(0)

真: 1 -5 'abc' [1,2] (1,) {one: 'one'} print

逻辑(布尔)运算符: not, and

- **not: 逻辑非** not expr (C语言等采用!表示逻辑非)
 - expr真值判断为真值时结果为False, expr真值判断为假值时结果为True
 - True: not False, not 0, not [], not (), not "", not range(0)
 - False: not True, not -1, not [1], not (1,), not 'hello', not range(1)
- **and: 逻辑与** expr1 and expr2 and expr3 比如n >= 0 and n < 10 或者 0 <= n < 10。 (C语言采用&&表示逻辑与)
 - 左结合律, 即从左到右进行运算
 - 只有所有条件(表达式)真值判断为真, 最终运算结果的真值判断才为真值
 - 短路逻辑(short-circuit logic) 或惰性求值(lazy evaluation)
 - 如果前面真值判断为假, 不管后面条件(表达式)怎样, 最终结果真值判断也是假值
 - 前面真值判断为假时, 后面表达式不需要计算

bool(expr1)	bool(expr2)	bool(expr1 and expr2)	expr1 and expr2
True	True	True	expr1为真, 取值为expr2
True	False	False	expr1为真, 取值为expr2
False	True	False	expr1为假, 取值为expr1
False	False	False	expr1为假, 取值为expr1

短路逻辑示例

短路逻辑(short-circuit logic) 或惰性求值(lazy evaluation)

- 如果前面真值判断为假，不管后面条件(表达式)怎样，最终结果真值判断也是假值
- 前面真值判断为假时，后面表达式不需要计算

```
if x != 0 and 100 / x > 2:  
    print('Great')
```

```
x = 40
```

```
if x > 10 and input('Print Value(y/n)?') == 'y':  
    print(x)
```

```
if x > 10 and ((answer := input('Print Value(y/n)?')) == 'y' or answer == 'Y'):  
    print(x)
```

```
# 最后一个例子使用了:=运算符
```

逻辑(布尔)运算符: and

expr1为真时expr2,否则 expr1

```
if expr1:
    expr = expr2
else:
    expr = expr1 # expr1真值为False
```

and :逻辑与 expr1 and expr2 and expr3 比如n>= 0 and n < 10

- 注意与其他语言不同, and/or运算得到的结果并不一定是True或者False, 而是**某个表达式的值**(用于if/while中的条件时再进行真值判断)
- and运算的结果为**最后一个决定最终的真值判断的表达式的值**, 即第一个假的表达式 (已经知道最终的真值判断为假值)或者最后一个表达式 (最终的真值判断由最后一个表达式决定)

bool(expr1)	bool(expr2)	bool(expr1 and expr2)	expr1 and expr2
True	True	True	expr1为真, 取值为expr2
True	False	False	expr1为真, 取值为expr2
False	True	False	expr1为假, 取值为expr1
False	False	False	expr1为假, 取值为expr1

```
>>> 4 and 5
5
>>> 4 and 0
0
>>> 0.0 and 5
0.0
>>> '' and 5
''
```

and : 返回**第一个假(None、空或者数值0)**的表达式或者最后一个表达式

```
name = input("please input your name: ")
prompt = name and f"{name}>>>"
```

name非空时, 使用**替代值**

逻辑(布尔)运算符: 逻辑或 or

- **and**: 逻辑与运算 `expr1 and expr2 and expr3`
 - 返回**第一个假**(None、空或者数值0)的表达式或者最后一个表达式
- **or**: 逻辑或运算 `expr1 or expr2 or expr3`
 - 采用左结合律, 即从左到右进行运算
 - 只要其中任一条件真值判断为真, 则最终结果的真值判断为真值, 否则为假值
 - 短路逻辑: 前面条件如果为真, 后面的表达式不需要进行计算
 - 返回**第一个真** (非None、非空或者非0) 的表达式或者最后一个表达式

bool(expr1)	bool(expr2)	bool(expr1 or expr2)	expr1 or expr2
True	True	True	expr1为真, expr1
True	False	True	expr1为真, expr1
False	True	True	expr1为假, expr2
False	False	False	expr1为假, expr2

```
if expr1:
    expr = expr1
else:
    expr = expr2  # expr1真值为False
```

```
name = input('Please enter your name: ') or '<unknown>'
```

- 对象为空对象(字符串, 列表等)采用**缺省值**

逻辑(布尔)运算符: if else

- if/else三元运算符: value1 if condition else value2

当条件表达式condition的真值判断为True时, 表达式的值为value1, 否则值为value2

- 短路逻辑: 不会同时对value1 和value2 求值
- 其他语言的三元表达式的语法一般为 condition?value1:value2

```
>>> x = 4
>>> x ** 2 if x % 2 == 0 else x ** 3
16
```

```
if condition:
    expr = value1
else:
    expr = value2
```

- 一元的not优先级最高, and第二, or优先级次之, 三元的ifelse优先级最低
 - expr1 and expr2 or expr3 相当于 (expr1 and expr2) or expr3
 - expr1 or expr2 and expr3 相当于 expr1 or (expr2 and expr3)
 - not expr1 and not expr2 相当于 (not expr1) and (not expr2)

运算符优先级

- 优先级越低，越最后计算
- 除了**为右结合外，其他运算符都是左结合
- `:= < lambda < if-else < 逻辑运算符(or, and, not) < 比较运算符 < 两元位运算 < 算术运算 < 一元运算 < 求幂运算 < 下标、切片、函数调用、字面量定义等`
 - 首先定义对象，然后才可以运算 → 字面量优先级最高
 - 函数调用、下标和切片、属性等返回对象 → 优先级次之
 - 有了对象，可以进行计算 → 算术运算符次之，一元的优先级更高
 - 计算后可以进行比较 → 比较运算符比算术运算符更低
 - 比较以及成员判断的结果为True/False，比较后可以进行逻辑运算 → 逻辑运算比较运算符低，单目逻辑运算符(not)更高
 - lambda表达式和赋值运算符优先级最低

从低到高

运算符	描述
<code>:= =</code>	海象(赋值)运算符
<code>lambda</code>	lambda表达式
<code>if-else</code>	3元条件表达式
<code>or</code>	布尔“或”
<code>and</code>	布尔“与”
<code>not x</code>	布尔“非”
<code>in, not in</code>	成员测试,
<code>is, is not</code>	同一性测试
<code><, <=, >, >=, !=, ==</code>	比较。所有比较类运算符优先级相同
<code> </code>	按位或
<code>^</code>	按位异或
<code>&</code>	按位与
<code><<, >></code>	移位
<code>+, -</code>	加法与减法
<code>*, /, //, %</code>	乘法、除法与取余
<code>+x, -x, ~x</code>	正负号, 按位求反, 一元运算或单目运算
<code>**</code>	求幂
<code>x.attribute</code>	属性
<code>x[index] x[index:index]</code>	下标和切片
<code>f(arguments...)</code>	函数调用
<code>(expression,...)</code>	元组字面量
<code>[expression,...]</code>	列表字面量
<code>{key:datum,...}</code>	字典字面量
<code>'expression,...'</code>	字符串字面量

逻辑运算符使用技巧

- 在设计条件表达式时,若能大概预测不同条件失败的概率或者根据表达式运算的开销,可将多个条件根据"and"和"or"运算的**短路特性**进行组织,提高程序运行效率
 - 表达式运算开销小的应该在前面进行求值
 - and运算时失败概率高, or运算时成功概率高的表达式应该在前面求值

- 德摩根定理(De Morgan's Law)**

$\text{not (expr1 and expr2) = not expr1 or not expr2}$

$\text{not (expr1 or expr2) = not expr1 and not expr2}$

```
i = 36
if i % 17 != 2 or i % 13 != 10:
    print(i, "不满足: 被17整除余2且被13整除余10")
else:
    print(i, "被17整除余2且被13整除余10")
```

conditional.py

```
i = 36
if not (i % 17 == 2 and i % 13 == 10):
    print(i, "不满足: 被17整除余2且被13整除余10")
else:
    print(i, "被17整除余2且被13整除余10")
```

```
if expr1 and expr2:
    blah blah...
else:
    pass
```

可以改写为:

```
if not expr1 or not expr2:
    pass
else:
    blah blah...
```

```
# if not (0 <= percent <= 100):
if percent < 0 or percent > 100:
    raise ValueError('invalid percent')
```

逻辑运算符使用技巧

- 分配律(distributive law)

expr1 and (expr2 or expr3) = (expr1 and expr2) or (expr1 and expr3)

expr1 or (expr2 and expr3) = (expr1 or expr2) and (expr1 or expr3)

```
(male == 'M' and age > 75) or (male == 'M' and age < 12)  
male == 'M' and (age > 75 or age < 12)
```

逻辑运算符使用技巧

问题：判断某一年是否闰年？

- 能被400整除
- 或者能被4整除，但不能被100整除

```
if year % 400 == 0 or year % 4 == 0 and year % 100 != 0:  
    leap = True  
else:  
    leap = False
```

```
leap = year % 400 == 0 or year % 4 == 0 and year % 100 != 0
```

逻辑运算符使用常见错误

- `if expr == value1 or expr == value2 or expr == value3:`

错误写成 `if expr == value1 or value2 or value3:`

- 类似地: `if expr < value1 or expr < value2:`

错误写成:

`if expr < value1 or value2 :`

优先级: 首先比, 然后逻辑运算, 相当于 `if (expr < value) or value2`

如果 `expr >= value1` 则 `value2` 的真值作为条件表达式的真值

`if expr < (value1 or value2):`

首先求解 `value1 or value2`, 然后与 `expr` 比较

主要内容

- 分支结构
 - 单分支/双分支/多分支结构
 - 逻辑运算符
- **python代码编写规范**

python代码编写规范

IDLE可通过下面方法进行代码块的缩进和反缩进:

- 菜单项: Format → Indent Region/Dedent Region
- 快捷键: Ctrl + [和 Ctrl +]

- 统一的编写规范:

- PEP 8 -- Style Guide for Python Code <https://www.python.org/dev/peps/pep-0008/>
- Google Python Style Guide <http://google.github.io/styleguide/pyguide.html>

- 代码布局(code layout)

- Python可以在同一行中放置多条语句, 语句之间使用分号 ";" 分割, 但为可读起见, **不建议在同一行中放置多条语句**
- **每行不超过79个字符** (不包括换行符), 如果一行语句太长, 可以在行尾加上\紧跟着换行, 这样可以分成多行, 但是更建议使用**括号**来包含多行内容
 - 分割行时选择在运算符之前断行
- python程序依靠代码块的**缩进**体现代码间的逻辑关系
 - def/if/while/for等复合语句中,**行尾的冒号 (后面可以包括空格等)** 表示下面应该是缩进的代码块
 - **缩进结束**表示代码块的结束
 - **同一个级别**的代码块的缩进量必须相同
- 建议以**4个空格为基本缩进单位**, 不建议采用制表符缩进, 更不能混合使用制表和空格

```
if a > b and a > c and a > 5 and b > 5 and c > 5 \
    and b > c:
    print('blah')

if (a > b and a > c and a > 5 and b > 5 and c > 5
    and b > c):
    print('blah')
```

python代码编写规范

空白符：空格、制表(Tab)和换行符

- 换行符：分割语句
- 缩进代码：具有特殊的含义
- 程序中可输入任意数量的空白符，建议遵循一个好的编程风格

- 代码布局(code layout)

- 必要的空行

- **不同功能**的代码块之间建议增加**一个空行**以提高可读性

- 不同的函数(包括class)定义之间使用**两个空行**隔开。class内部的方法之间以**一个空行**隔开

- 必要的空格：行尾不要有多余的空格

- 建议在**逗号后面**添加一个空格，但是逗号前面没有空格

- 函数定义时的缺省值参数以及调用时的关键字参数中**=前后**没有空格

- **赋值语句的=、比较运算符以及逻辑运算符**(and or not) 的前后各一个空格

- **算术运算符**

- 如果只有一个算术运算符，运算符两侧前后各一个空格

- 如果有多个优先级的运算符出现时，**更低优先级的运算符**前后添加一个空格

```
def func(a, b, c, x='x', y='y'):
    print(a, b, c, x, y)
```

```
func(1, 2, 3, y=5)
```

```
a = 4 + 5
```

```
x = x*2 - 1
```

```
hypot2 = x*x + y*y    # +优先级更低
```

```
c = (a+b) * (a-b)    # *相比括号优先级更低
```

python代码编写规范

- 一个好的、可读性强的程序应该有注释
 - 以#开始（#后面建议有一个空格）表示本行**#之后的内容为注释**
 - 在模块开始和函数体开始处可添加长注释（三引号），作为**文档字符串**，文档字符串同样也与代码块有相同的缩进
- 每个import只导入一个模块
 - 首先导入Python标准库模块，如os、sys、re
 - 导入第三方扩展库，如numpy、scipy
 - 导入自己实现的本地模块
- 命名规则：参见前面介绍赋值语句时建议变量名的命名规范
 - 建议采用ASCII字符集中的字符（字母数字和下划线），采用有意义的名字
 - 模块、函数、方法、变量等采用小写字母，如果为多个单词时，单词间以下划线隔开
 - 类型名建议采用CamelCase(驼峰) 名，即每个单词首字母大写，比如CapWords
 - 常量一般在模块级定义，建议采用大写形式，比如MAX_OVERFLOW、TOTAL
 - package采用小写字母，单词之间不进行分割，比如matplotlib

IDLE开发环境中，可快速注释/解除注释大段内容：

- Format→Comment Out Region/Uncomment Region
- 快捷键： ALT+3 ALT+4

python代码编写规范

- linter: 来源于早期UNIX命令lint, 静态代码分析, 用来检查可能的编程错误、安全缺陷、编程风格
 - Python语言的Linter工具包括pycodestyle, flake8, pylint, **pylance**, ruff等
- formatter: 格式化代码
 - Python语言的格式化工具包括: autopep8, yapf, **black和ruff**等